

دليل تقني عملي شامل

**Claude**

من الصفر إلى الاحتراف

A Practical Guide to Claude, Anthropic, Claude Code, the API, MCP,  
and AI Agents

**ANTHROPIC · CLAUDE CODE · API · MCP · AGENTS**

الإصدار الأول — أغسطس 2026

مرجع تعليمي مستقل — غير صادر عن شركة Anthropic

# Claude

## من الصفر إلى الاحتراف

A Practical Guide to Claude, Anthropic, Claude Code, the API, MCP,  
and AI Agents

إعداد: تسنيم نمر نصر

الإصدار الأول — أغسطس 2026

مرجع تعليمي مستقل — غير صادر عن شركة Anthropic

## مقدمة الناشر

هذا الكتاب مرجع عملي مستقل حول Claude، النموذج اللغوي الذي تطوره شركة Anthropic، وحول الأدوات المحيطة به: واجهة المحادثة، Claude Code، API، بروتوكول Model MCP Context Protocol، وبناء الوكلاء الذكيين (Agents). الهدف منه ليس تكرار الوثائق الرسمية، بل تقديم مسار تعلم متكامل يبدأ من المفاهيم الأساسية وينتهي بمشاريع حقيقية قابلة للتطبيق.

الكتاب موجّه لثلاث فئات رئيسية: المبرمج الذي يريد دمج Claude في سير عمله اليومي، طالب هندسة البرمجيات الذي يتعلّم أدوات الذكاء الاصطناعي الحديثة، والمطور المحترف الذي يبني تطبيقات وأنظمة تعتمد على API أو MCP بشكل مباشر.

نماذج Claude والأدوات المرتبطة بها تتطور بوتيرة سريعة؛ الإصدارات وأسماء النماذج والميزات قد تتغير بعد صدور هذا الكتاب. لهذا السبب حرصنا على الإشارة إلى الوثائق الرسمية في كل فصل، بحيث يبقى الكتاب نقطة انطلاق صحيحة حتى بعد تحديث المنتجات.

♦ ملاحظة: جميع الروابط الرسمية في هذا الكتاب تشير إلى مصادر Anthropic المباشرة (docs.claude.com و anthropic.com). يُنصح بالرجوع إليها للتحقق من أي تفاصيل حساسة للوقت مثل الأسعار أو حدود الاستخدام.

## جدول المحتويات

◆ هذا الفهرس تفاعلي: في Microsoft Word، انقر بزر الفأرة الأيمن على الفهرس واختر Update Field (تحديث الحقل) لبناء الروابط، ثم يمكن الضغط مع مفتاح Ctrl على أي عنوان للانتقال مباشرة إليه. [Claude-من-الصفر-إلى-الاحتراف-نسخة-منسقة.docx](#)

|                                                 |    |
|-------------------------------------------------|----|
| 1. مقدمة الناشر.....                            | 1  |
| 2. جدول المحتويات.....                          | 2  |
| 9. مقدمة الكتاب.....                            | 9  |
| 10. الفصل 1 - مقدمة إلى Claude و Anthropic..... | 10 |
| ما هو Claude؟.....                              | 10 |
| ما هي Anthropic؟.....                           | 10 |
| تاريخ وتطور Claude.....                         | 11 |
| نماذج Claude.....                               | 12 |
| متى أستخدم Claude؟.....                         | 13 |
| 14. الفصل 2 - فهم نماذج Claude.....             | 14 |
| عائلات النماذج.....                             | 14 |
| نافذة السياق (Context Window).....              | 14 |
| الـ Tokens.....                                 | 15 |
| الاستدلال (Reasoning) والتفكير الممتد.....      | 15 |
| القدرات متعددة الوسائط (Multimodal).....        | 15 |
| اختيار النموذج المناسب للمهمة.....              | 15 |
| 16. الفصل 3 - البدء مع Claude.....              | 16 |
| إنشاء الحساب.....                               | 16 |

|                                                      |    |
|------------------------------------------------------|----|
| Claude واجهة.....                                    | 16 |
| Projects .....                                       | 16 |
| Files و Conversations.....                           | 17 |
| إعداد بيئة العمل.....                                | 17 |
| أهم الإعدادات.....                                   | 17 |
| الفصل 4 - هندسة الأوامر (Prompt Engineering).....    | 18 |
| ما هو Prompt Engineering؟.....                       | 18 |
| عناصر الأمر الجيد (Anatomy of a Good Prompt).....    | 18 |
| مثال عملي.....                                       | 19 |
| Few-shot Prompting.....                              | 19 |
| Chain of Thought بشكل آمن ومناسب.....                | 19 |
| قوالب الأوامر (Prompt Templates).....                | 20 |
| الفصل 5 - تقنيات متقدمة في هندسة الأوامر.....        | 21 |
| الأوامر المهيكلية وعلامات XML.....                   | 21 |
| أمثلة Few-shot متقدمة.....                           | 21 |
| المراجعة الذاتية (Self-checking).....                | 22 |
| الأوامر التكرارية وتحسين الأوامر.....                | 22 |
| العمل مع السياق الطويل (Long-context Prompting)..... | 22 |
| التعامل مع المهام المعقدة.....                       | 22 |
| الفصل 6 - Claude في البرمجة.....                     | 23 |
| كتابة الكود.....                                     | 23 |
| شرح الكود.....                                       | 23 |
| Debugging .....                                      | 23 |
| إعادة الهيكلة (Refactoring).....                     | 23 |
| مراجعة الكود (Code Review).....                      | 24 |

|                                               |           |
|-----------------------------------------------|-----------|
| الاختبار (Testing) .....                      | 24        |
| التوثيق (Documentation) .....                 | 24        |
| التعامل مع مشاريع كاملة .....                 | 24        |
| <b>الفصل 7 · Claude Code</b> .....            | <b>25</b> |
| ما هو Claude Code ؟ .....                     | 25        |
| التثبيت (Installation) .....                  | 25        |
| الإعداد الأولي (Setup) .....                  | 26        |
| أهم الأوامر داخل الجلسة التفاعلية .....       | 27        |
| أهم أوامر سطر النظام (خارج الجلسة) .....      | 27        |
| ملف CLAUDE.md .....                           | 27        |
| سير عمل المشروع وفهم بنية الكود .....         | 28        |
| سير عمل Git .....                             | 28        |
| الاختبار والأتمتة .....                       | 28        |
| أفضل الممارسات .....                          | 28        |
| <b>الفصل 8 · Claude API</b> .....             | <b>29</b> |
| أساسيات الـ API .....                         | 29        |
| مفاتيح الـ API والمصادقة .....                | 29        |
| بنية الطلب والاستجابة .....                   | 29        |
| أهم المعاملات (Parameters) .....              | 30        |
| تعليمات النظام (System Prompts) .....         | 30        |
| البث (Streaming) .....                        | 31        |
| معالجة الأخطاء .....                          | 31        |
| حدود الاستخدام (Rate Limits) .....            | 31        |
| <b>الفصل 9 · استخدام الـ API عمليًا</b> ..... | <b>32</b> |
| Python .....                                  | 32        |

|                                                                |    |
|----------------------------------------------------------------|----|
| JavaScript / TypeScript.....                                   | 33 |
| REST مباشر مقابل SDK .....                                     | 33 |
| بناء تطبيق بسيط.....                                           | 33 |
| التعامل مع الملفات.....                                        | 34 |
| المخرجات المهيكلة (Structured Outputs).....                    | 34 |
| اعتبارات الإنتاج (Production Considerations).....              | 34 |
| الفصل 10 - استخدام الأدوات (Tool Use).....                     | 35 |
| ما هو Tool Use؟.....                                           | 35 |
| كيف تعمل الآلية؟.....                                          | 35 |
| تعريف الأدوات (Tool Definitions).....                          | 35 |
| استدعاءات الأدوات (Tool Calls).....                            | 36 |
| مثال عملي مبسط.....                                            | 36 |
| متى يُستخدم Tool Use؟.....                                     | 36 |
| الفصل 11 - بروتوكول Model Context Protocol (MCP).....          | 37 |
| ما هو MCP؟.....                                                | 37 |
| لماذا تم تطويره؟.....                                          | 37 |
| البنية العامة (Architecture).....                              | 38 |
| كيفية استخدام MCP.....                                         | 38 |
| أمثلة وحالات استخدام حقيقية.....                               | 38 |
| الفصل 12 - وكلاء Claude والعمل الوكيل (Agentic Workflows)..... | 40 |
| ما هو AI Agent؟.....                                           | 40 |
| Agent مقابل Chatbot.....                                       | 40 |
| حلقة عمل الوكيل (Agent Loop).....                              | 40 |
| التخطيط والذاكرة.....                                          | 41 |
| بناء وكيل بسيط.....                                            | 41 |

|                                                    |    |
|----------------------------------------------------|----|
| أفضل الممارسات.....                                | 41 |
| الفصل 13 · Claude في هندسة البرمجيات.....          | 42 |
| تصميم البنية البرمجية (Software Architecture)..... | 42 |
| تحليل المتطلبات.....                               | 42 |
| UML وتصميم قواعد البيانات.....                     | 42 |
| Backend و Frontend و Mobile.....                   | 42 |
| الاختبار و CI/CD.....                              | 43 |
| مراجعة الكود وسير عمل GitHub.....                  | 43 |
| التوثيق التقني للمشاريع.....                       | 43 |
| الفصل 14 · Claude للطلاب والتعلم.....              | 44 |
| الدراسة باستخدام Claude.....                       | 44 |
| فهم المحاضرات وتلخيص الكتب.....                    | 44 |
| حل المسائل.....                                    | 44 |
| تعلم البرمجة.....                                  | 44 |
| إنشاء خطة دراسة.....                               | 44 |
| الاستعداد للمقابلات التقنية.....                   | 45 |
| الفصل 15 · Claude للإنتاجية.....                   | 46 |
| الكتابة.....                                       | 46 |
| البحث والتلخيص.....                                | 46 |
| تحليل البيانات.....                                | 46 |
| البريد الإلكتروني والتقارير.....                   | 46 |
| العصف الذهني والتخطيط.....                         | 46 |
| الأتمة.....                                        | 47 |
| الفصل 16 · Claude في المحتوى والعمل الإبداعي.....  | 48 |
| الكتابة الإبداعية.....                             | 48 |



|                                                             |    |
|-------------------------------------------------------------|----|
| إنشاء المحتوى ومنصات التواصل.....                           | 48 |
| التسويق .....                                               | 48 |
| التحرير والمراجعة.....                                      | 48 |
| سير العمل الإبداعي.....                                     | 48 |
| <b>الفصل 17 - سير عمل متقدمة مع Claude</b> .....            | 49 |
| المهام متعددة الخطوات.....                                  | 49 |
| سلسلة الأوامر (Prompt Chaining).....                        | 49 |
| Claude مع الأدوات الخارجية والـ APIs.....                   | 49 |
| Claude و GitHub.....                                        | 49 |
| أمثلة سير عمل شامل.....                                     | 50 |
| <b>الفصل 18 - الأمان والخصوصية والاستخدام المسؤول</b> ..... | 51 |
| الخصوصية والتعامل مع البيانات.....                          | 51 |
| أمان مفاتيح الـ API.....                                    | 51 |
| حقن الأوامر (Prompt Injection).....                         | 51 |
| المعلومات الحساسة.....                                      | 52 |
| أفضل ممارسات الأمان.....                                    | 52 |
| الاستخدام المسؤول للذكاء الاصطناعي.....                     | 52 |
| <b>الفصل 19 - مشاريع تطبيقية واقعية</b> .....               | 53 |
| مشروع 1 — مساعد برمجي داخل الطرفية.....                     | 53 |
| مشروع 2 — محلّ مستندات.....                                 | 53 |
| مشروع 3 — مساعد دراسي تفاعلي.....                           | 53 |
| مشروع 4 — تطبيق مبني على الـ API بالكامل.....               | 54 |
| مشروع 5 — تكامل عبر MCP.....                                | 54 |
| مشروع 6 — وكيل أتمتة سير عمل.....                           | 54 |
| <b>الفصل 20 - خارطة طريق احترافية مع Claude</b> .....       | 55 |

---

|                                     |    |
|-------------------------------------|----|
| كيف تصبح محترفاً في استخدام Claude؟ | 55 |
| مسار التعلم المقترح                 | 55 |
| تطوير المهارات عبر المشاريع         | 55 |
| مصادر التعلم المستمر                | 56 |
| بناء معرض أعمال (Portfolio)         | 56 |
| فرص العمل المرتبطة                  | 56 |
| الخطوات التالية                     | 56 |
| خاتمة                               | 57 |
| المراجع والمصادر الرسمية            | 58 |

## مقدمة الكتاب

صدر **Claude** لأول مرة في مارس 2023، وتطوّر منذ ذلك الحين عبر عدة أجيال من النماذج حتى أصبح اليوم أحد الأنظمة اللغوية الأساسية المستخدمة في تطوير البرمجيات والبحث والإنتاجية اليومية. الفارق بين استخدام **Claude** كأداة محادثة بسيطة واستخدامه كجزء من سير عمل هندسي متكامل هو فهم بنيته وأدواته وحدوده.

يتناول هذا الكتاب أربعة محاور رئيسية:

- الأساسيات: ما هو **Claude**، عائلات النماذج، وكيفية اختيار النموذج المناسب لكل مهمة.
  - الاستخدام التفاعلي: هندسة الأوامر (**Prompt Engineering**)، واجهة المحادثة، والمشاريع.
  - الأدوات البرمجية: **Tool Use**، **Claude Code**، **API**، و**MCP**.
  - التطبيق العملي: الوكلاء الذكيون، مشاريع واقعية، وخارطة طريق احترافية.
- كل فصل مستقل بذاته، لكن الفصول مرتبة بحيث يبني كل فصل على ما سبقه. القارئ الذي يريد مرجعًا سريعًا يمكنه الانتقال مباشرة إلى الفصل المطلوب عبر الفهرس التفاعلي؛ أما من يريد مسارًا تعليميًا كاملاً فمن الأفضل قراءة الفصول بالترتيب.

## الفصل 1 - مقدمة إلى Claude و Anthropic

### ما هو Claude؟

Claude هو عائلة من النماذج اللغوية الكبيرة (Large Language Models) التي تطورها شركة Anthropic. يُستخدم Claude لفهم اللغة الطبيعية وتوليدها، وتحليل الصور والمستندات، وكتابة الأكواد البرمجية ومراجعتها، والعمل كوكيل (Agent) قادر على استخدام أدوات خارجية لإنجاز مهام متعددة الخطوات.

يمكن الوصول إلى Claude عبر عدة قنوات: واجهة المحادثة على [claude.ai](https://claude.ai) وتطبيقات الجوال وسطح المكتب، أو برمجياً عبر Claude API، أو من خلال أداة سطر الأوامر Claude Code، أو عبر تكاملات مثل Claude in Chrome و Claude in Excel و Claude Cowork و Claude Tag على Slack.

### ما هي Anthropic؟

Anthropic شركة أبحاث وتطوير في مجال الذكاء الاصطناعي، تأسست عام 2021 على يد مجموعة من الباحثين السابقين في OpenAI، من بينهم داريو أموداي وشقيقته دانييلا أموداي. تتمحور فلسفة الشركة حول تطوير أنظمة ذكاء اصطناعي قوية وآمنة في آن واحد، وهو ما ينعكس في منهجها البحثي المعروف باسم Constitutional AI، والذي يهدف إلى تدريب النماذج على اتباع مجموعة من المبادئ الأخلاقية والسلوكية بدلاً من الاعتماد الكامل على التغذية الراجعة البشرية المباشرة فقط.

➔ [الموقع الرسمي لشركة Anthropic](https://www.anthropic.com)

<https://www.anthropic.com>

## تاريخ وتطور Claude

أُطلق أول إصدار من Claude للعامة في مارس 2023. منذ ذلك الحين مرّت النماذج بعدة أجيال رئيسية: Claude 1 و Claude 2 في 2023، ثم عائلة Claude 3 في أوائل 2024 التي أدخلت التصنيف الثلاثي المعروف حتى اليوم (Haiku للسرعة، Sonnet للتوازن، Opus للقدرة القصوى)، تلتها تحديثات Claude 3.5 و Claude 3.7 التي أضافت قدرات Extended Thinking واستخدام الأدوات المتقدم. جيل Claude 4 وتحديثاته الفرعية (x.4) رسّخ استخدام Claude في العمل الوكيل (Agentic) طويل المدى وفي بيئات الترميز الاحترافية.

في يونيو 2026 قدّمت Anthropic فئة جديدة أعلى من Opus تُعرف باسم Mythos، ومعها أول نموذجين من هذه الفئة: Claude Fable 5 و Claude Mythos 5. يشترك النموذجان في نفس الأساس، إلا أن Fable 5 يتضمن طبقات أمان إضافية في مجالات الأحياء والأمن السيبراني وأبحاث النماذج اللغوية، وهو النموذج المتاح للجمهور، بينما يبقى Mythos 5 محصورًا حاليًا ضمن برنامج Project Glasswing لعدد محدود من الجهات الموثوقة.

♦ بسبب ضوابط تصدير أمريكية، عُلّق الوصول إلى Claude Fable 5 و Claude Mythos 5 مؤقتًا بين 12 و 30 يونيو 2026 ثم أُعيد تفعيله في 1 يوليو 2026. للتفاصيل الرسمية: انظر بيان Anthropic على [anthropic.com/news/fable-mythos-access](https://anthropic.com/news/fable-mythos-access).

## Claude نماذج

تنظم Anthropic نماذجها الحالية في أربع فئات رئيسية، بالإضافة إلى الفئة المحدودة الوصول:

| النموذج             | الفئة                        | الاستخدام الأساسي                                      |
|---------------------|------------------------------|--------------------------------------------------------|
| Claude Haiku<br>4.5 | الأسرع والأقل تكلفة          | مهام التصنيف والاستجابة السريعة<br>وحجم الطلبات الكبير |
| Claude Sonnet<br>5  | متوازن                       | الاستخدام اليومي، البرمجة،<br>والوكلاء المتوسطون       |
| Claude Opus<br>4.8  | الأعلى قدرة ضمن الفئة العامة | المهام المعقدة والاستدلال العميق<br>والمشاريع الطويلة  |
| Claude Fable 5      | فئة Mythos العامة            | أعلى قدرة متاحة للجمهور، مع<br>طبقات أمان إضافية       |
| Claude Mythos<br>5  | فئة Mythos محدودة الوصول     | متاح فقط ضمن Project<br>Glasswing                      |

أسماء النماذج في الـ (API (model strings تُستخدم حرفياً عند إرسال الطلبات، مثل claude-sonnet-5 و claude-opus-4 و claude-haiku-4 و claude-fable-5. لأن هذه القائمة تتغير مع كل إصدار جديد، يجب دائماً التحقق من القائمة الرسمية قبل البدء بمشروع إنتاجي.

➔ [قائمة النماذج الرسمية والمحدثة](https://docs.claude.com/en/docs/about-claude/models/overview)

<https://docs.claude.com/en/docs/about-claude/models/overview>

## متى أستخدم Claude؟

اختيار الأداة المناسبة يعتمد على طبيعة المهمة أكثر من اعتماده على تفضيل شخصي. الجدول التالي يلخص أكثر السيناريوهات شيوعاً:

| السيناريو                                     | الأداة المناسبة                  |
|-----------------------------------------------|----------------------------------|
| سؤال سريع أو مساعدة في الكتابة                | واجهة Claude على الويب أو الجوال |
| العمل داخل مستودع كود موجود                   | Claude Code                      |
| بناء تطبيق يستدعي Claude برمجياً              | Claude API                       |
| ربط Claude بأدوات وأنظمة خارجية               | Model Context Protocol (MCP)     |
| أتمتة مهام متعددة الخطوات دون تدخل بشري مستمر | Agents / Agentic workflows       |

## الفصل 2 - فهم نماذج Claude

### عائلات النماذج

كل نموذج من نماذج Claude يمثل نقطة توازن مختلفة بين ثلاثة عوامل: سرعة الاستجابة، جودة الاستدلال، والتكلفة لكل مليون Token. فهم هذا التوازن هو أساس اختيار النموذج الصحيح، بدلاً من افتراض أن النموذج الأقوى هو الخيار الأفضل دائماً.

### الفرق بين النماذج

- **Haiku:** زمن استجابة منخفض جداً، مناسب للمهام المتكررة وواسعة النطاق مثل التصنيف والتلخيص القصير.
- **Sonnet:** يغطي غالبية الاستخدامات اليومية بما فيها البرمجة والتحليل متوسط التعقيد، وهو غالباً الخيار الافتراضي المعقول.
- **Opus:** مخصص للمهام التي تتطلب استدلالاً عميقاً أو سياقاً طويلاً أو دقة عالية جداً، بتكلفة أعلى وزمن استجابة أطول.
- **Fable 5 / Mythos:** أعلى قدرة متاحة، مخصصة للمهام شديدة التعقيد أو البحثية التي تبرر التكلفة الإضافية.

### نافذة السياق (Context Window)

نافذة السياق هي الحد الأقصى لكمية النص (مُقاسة بالـ Tokens) التي يمكن لنموذج Claude معالجتها في طلب واحد، متضمنة الرسائل السابقة والملفات المرفقة والتعليمات والرد المتوقع. النماذج الحديثة من Claude تدعم نوافذ سياق كبيرة تصل إلى مئات الآلاف من Tokens وحتى مليون Token في بعض النماذج والخطط، مما يسمح بمعالجة مستندات طويلة أو قواعد أكواد كاملة في طلب واحد.



◆ **حجم نافذة السياق الفعلي يختلف حسب النموذج وخطة الاشتراك أو مستوى الوصول في الـ API.**  
يجب التحقق من الرقم الدقيق في وثائق النموذج قبل تصميم أي نظام يعتمد عليه.

## الـ Tokens

**Token** هو وحدة النص الأساسية التي يتعامل معها النموذج، وقد تمثل كلمة كاملة أو جزءًا من كلمة أو حتى علامة ترقيم. في اللغة العربية، عدد الـ Tokens لكل كلمة يكون عادة أعلى منه في الإنجليزية بسبب طريقة تقطيع النص، وهو ما يجب أخذه بعين الاعتبار عند تقدير التكلفة أو حدود نافذة السياق لمحتوى عربي طويل.

## الاستدلال (Reasoning) والتفكير الممتد

تدعم نماذج **Claude** الحديثة وضع **Extended Thinking**، حيث يقوم النموذج بعملية استدلال داخلية أطول قبل تقديم الإجابة النهائية، وهو مفيد بشكل خاص في المسائل الرياضية والمنطقية والمهام التي تتطلب تخطيطًا متعدد الخطوات. هذا الوضع يزيد من زمن الاستجابة والتكلفة، لذلك يُستخدم بشكل انتقائي وليس كأعداد افتراضي لكل طلب.

## القدرة متعددة الوسائط (Multimodal)

يمكن لنماذج **Claude** معالجة الصور والمستندات (مثل ملفات PDF) إلى جانب النصوص، وتحليل محتواها والإجابة عن أسئلة تتعلق بها أو استخراج بيانات منها. هذه القدرة مفيدة في تحليل المخططات التقنية، مراجعة تصاميم الواجهات، أو استخراج جداول من مستندات ممسوحة ضوئيًا.

## اختيار النموذج المناسب للمهمة

قاعدة عملية شائعة: ابدأ بنموذج **Sonnet** لمعظم المهام. إذا كانت النتائج غير كافية من حيث الدقة أو عمق الاستدلال، انتقل إلى **Opus** أو **Fable**. إذا كانت المهمة بسيطة ومتكررة بكميات كبيرة، استخدم **Haiku** لتقليل التكلفة وزمن الاستجابة.

## الفصل 3 - البدء مع Claude

### إنشاء الحساب

يبدأ الوصول إلى Claude بإنشاء حساب عبر [claude.ai](https://claude.ai)، باستخدام البريد الإلكتروني أو حساب Google. تتوفر خطط متعددة (مجانية ومدفوعة) تختلف في النماذج المتاحة وحدود عدد الرسائل والميزات الإضافية. لأن تفاصيل الخطط وحدودها تتغير بشكل متكرر، يُفضل التحقق منها مباشرة من مركز مساعدة Anthropic.

[مركز مساعدة Claude](https://support.claude.com) ↗

<https://support.claude.com>

### واجهة Claude

واجهة المحادثة متاحة كتطبيق ويب، وتطبيقات جوال لنظامي iOS وAndroid، وتطبيق سطح مكتب. تحافظ الواجهة على نفس المحادثات والإعدادات عبر الأجهزة عند تسجيل الدخول بنفس الحساب، ويمكن التبديل بين النماذج المتاحة من داخل نفس المحادثة.

### Projects

تسمح ميزة Projects بتجميع محادثات متعددة ترتبط بسياق عمل واحد، مع إمكانية إرفاق تعليمات ثابتة وملفات مرجعية تبقى متاحة لكل محادثة جديدة داخل المشروع. هذا مفيد عند العمل على مهمة ممتدة مثل تطوير ميزة برمجية أو كتابة تقرير طويل، حيث لا حاجة لإعادة شرح السياق في كل مرة.

## Files و Conversations

كل محادثة (Conversation) مستقلة بذاكرتها ضمن الجلسة، ويمكن إرفاق ملفات نصية أو صور أو مستندات PDF أو جداول بيانات مباشرة داخل المحادثة ليقوم Claude بتحليلها. الملفات المرفقة تُحتسب ضمن نافذة السياق، لذا فإن إرفاق مستندات ضخمة جدًا قد يستهلك جزءًا كبيرًا من الحد المتاح للنموذج.

### إعداد بيئة العمل

بالنسبة للمطورين الذين يخططون لاستخدام Claude Code أو الـ API، يُنصح بإعداد بيئة عمل تحتوي على Node.js أو Python (بحسب لغة المشروع)، ومفتاح API صالح من Anthropic Console، وأداة إدارة إصدارات مثل Git. الفصلان 7 و8 يغطيان هذا الإعداد بالتفصيل.

### أهم الإعدادات

- اختيار النموذج الافتراضي للمحادثات الجديدة.
- تفعيل أو تعطيل خاصية البحث على الويب (Web Search) حسب الحاجة.
- إدارة الذاكرة والتفضيلات الشخصية إذا كانت متاحة في الحساب.
- ضبط أسلوب الاستجابة (Style) لتخصيص نبرة وطول الردود.

## الفصل 4 - هندسة الأوامر (Prompt Engineering)

### ما هو Prompt Engineering؟

**Prompt Engineering** هو عملية صياغة التعليمات الموجهة للنموذج بطريقة تزيد من احتمال الحصول على استجابة دقيقة ومفيدة ومطابقة للهدف المطلوب. لا يتعلق الأمر بـ'خداع' النموذج أو استخدام حيل لغوية، بل بتوضيح السياق والهدف والقيود بشكل كافٍ، تمامًا كما تفعل عند تكليف زميل عمل بمهمة يفتقر لخلفية كاملة عنها.

### عناصر الأمر الجيد (Anatomy of a Good Prompt)

الأمر الفعال عادة ما يتضمن مزيجًا من العناصر التالية، دون أن يعني ذلك وجوب استخدامها كلها في كل مرة:

| العنصر        | الوظيفة                                                             |
|---------------|---------------------------------------------------------------------|
| Role          | تحديد الدور الذي يجب أن يتبناه النموذج، مثل مراجع كود أو محرر تقني. |
| Context       | المعلومات الخلفية اللازمة لفهم المهمة بشكل صحيح.                    |
| Instructions  | الوصف الدقيق للمطلوب إنجازه.                                        |
| Constraints   | القيود مثل الطول أو الأسلوب أو ما يجب تجنبه.                        |
| Examples      | أمثلة توضح الشكل أو الجودة المطلوبة (Few-shot).                     |
| Output format | الصيغة المطلوبة للناتج، مثل JSON أو جدول أو قائمة نقطية.            |

## مثال عملي

**Role:** You are a senior backend code reviewer.

**Context:** The following function handles payment retries in a Node.js service.

**Instructions:** Review the code for race conditions and error handling gaps.

**Constraints:** Keep the review under 200 words. Do not rewrite the whole function.

**Output format:** A bullet list of issues, each with severity (low/medium/high).

لاحظ أن هذا الأمر مكتوب بالإنجليزية رغم أن الكتاب باللغة العربية؛ هذا مقصود، إذ إن كتابة الأوامر التقنية بالإنجليزية غالبًا ما تعطي نتائج أكثر اتساقًا عند العمل على كود أو وثائق تقنية إنجليزية، بينما تبقى العربية الخيار الأنسب عند التعامل مع محتوى أو جمهور عربي.

## Few-shot Prompting

تقنية **Few-shot** تقوم على تزويد النموذج بأمثلة قليلة (عادة من واحد إلى خمسة) توضح نمط الإدخال والإخراج المطلوب قبل تقديم الطلب الفعلي. هذه التقنية فعالة بشكل خاص عندما يكون الشكل المطلوب للنتائج غير قياسي أو حين يصعب وصفه بدقة عبر تعليمات نصية فقط.

## Chain of Thought

تحسين جودة الاستدلال أحيانًا يتطلب حث النموذج على "التفكير خطوة بخطوة" قبل الوصول إلى النتيجة النهائية، خصوصًا في المسائل المنطقية أو الحسابية المتعددة الخطوات. النماذج الحديثة من **Claude** التي تدعم **Extended Thinking** تقوم بهذا داخليًا دون الحاجة لصياغة يدوية معقدة، لكن في حالات أبسط يمكن ببساطة طلب توضيح خطوات الحل قبل الإجابة النهائية.

## قوالب الأوامر (Prompt Templates)

بناء قوالب أوامر قابلة لإعادة الاستخدام يوفر الوقت ويحسن الاتساق، خصوصاً في المهام المتكررة مثل مراجعة الأكواد أو تلخيص التقارير. القالب الجيد يترك الأجزاء المتغيرة (مثل النص المدخل) في مكان واضح، بينما تبقى التعليمات والقيود ثابتة.

## الفصل 5 - تقنيات متقدمة في هندسة الأوامر

### الأوامر المهيكلية وعلامات XML

عند التعامل مع أوامر طويلة تحتوي على أجزاء متعددة (سياق، بيانات، تعليمات)، يساعد استخدام علامات تشبه XML في فصل هذه الأجزاء بوضوح، مما يقلل من التباس النموذج حول أي جزء من النص هو تعليمات وأيها بيانات يجب معالجتها فقط.

```
<context>
```

```
The following is a customer support transcript.
```

```
</context>
```

```
<transcript>
```

```
...
```

```
</transcript>
```

```
<instructions>
```

```
Summarize the customer's main complaint in one sentence.
```

```
</instructions>
```

### أمثلة Few-shot متقدمة

في المهام الأكثر دقة، مثل تصنيف نصوص بمعايير دقيقة، يفضل استخدام أمثلة تغطي الحالات الحدية (Edge cases) وليس فقط الحالات الواضحة، لأن هذه الحالات هي التي يخطئ فيها النموذج غالبًا دون توجيه إضافي.

## المراجعة الذاتية (Self-checking)

يمكن تحسين دقة النتائج بمطالبة النموذج بمراجعة إجابته الخاصة مقابل معايير محددة قبل تقديمها بشكل نهائي، مثل التحقق من اتساق الأرقام في تقرير مالي أو التأكد من أن الكود المكتوب يغطي الحالات المذكورة في المتطلبات.

### الأوامر التكرارية وتحسين الأوامر

نادرًا ما يكون أول أمر مكتوب هو الأمثل. الأسلوب العملي هو البدء بأمر بسيط وواضح، مراقبة النتيجة، ثم تعديل الأمر بناءً على أوجه القصور الملاحظة، سواء كانت متعلقة بالتنسيق أو العمق أو الالتزام بالقيود.

## العمل مع السياق الطويل (Long-context Prompting)

عند تزويد النموذج بمستندات طويلة، يُفضل وضع التعليمات النهائية بعد المحتوى وليس قبله، لأن ذلك يزيد من احتمال أن يعالج النموذج التعليمات في ضوء كامل السياق الذي قرأه للتو. كما يُفضل تقسيم المستندات الكبيرة جدًا إلى أجزاء منطقية عند الحاجة لمعالجة تفصيلية لكل جزء.

### التعامل مع المهام المعقدة

المهام المعقدة التي تتضمن عدة خطوات مترابطة (مثل تحليل بيانات ثم كتابة تقرير ثم اقتراح توصيات) غالبًا ما تُنجز بجودة أعلى عند تقسيمها إلى سلسلة من الأوامر المتتالية (Prompt Chaining) بدلاً من طلب واحد ضخم يحاول تغطية كل شيء دفعة واحدة. هذا المفهوم يُفصّل أكثر في الفصل السابع عشر.



## الفصل 6 - Claude في البرمجة

### كتابة الكود

يمكن لـ Claude كتابة كود جديد بلغات برمجة متعددة انطلاقاً من وصف نصي للمتطلبات. جودة الكود الناتج تتحسن بشكل ملحوظ عندما يتضمن الطلب: لغة البرمجة والإطار المستخدم، نمط الكود المتبع في المشروع، والقيود مثل التوافق مع إصدار معين أو تجنب مكتبات خارجية إضافية.

### شرح الكود

عند التعامل مع كود غير مألوف أو قديم، يمكن لـ Claude شرح منطق الكود سطرًا بسطر أو تلخيص وظيفة ملف كامل. هذا مفيد بشكل خاص عند الانضمام إلى مشروع موجود أو عند مراجعة كود كتبه فريق آخر.

## Debugging

لتشخيص خطأ برمجي بفعالية، يُفضّل تزويد Claude برسالة الخطأ كاملة، والكود المرتبط بها، ووصف السلوك المتوقع مقابل السلوك الفعلي. الأمر المبهم مثل "لماذا لا يعمل هذا الكود؟" بدون سياق كافٍ ينتج تخمينات بدلاً من تشخيص دقيق.

## إعادة الهيكلة (Refactoring)

عند طلب إعادة هيكلة كود موجود، حدّد الهدف بوضوح: تحسين القابلية للقراءة، تقليل التكرار، تحسين الأداء، أو تسهيل الاختبار. إعادة الهيكلة دون هدف محدد قد تنتج تغييرات غير ضرورية تزيد من صعوبة المراجعة.

## مراجعة الكود (Code Review)

يمكن استخدام **Claude** كمراجع أولي للكود قبل تقديمه لمراجعة بشرية، للكشف عن مشاكل شائعة مثل معالجة الأخطاء الناقصة، الثغرات الأمنية البسيطة، أو مخالفة معايير الأسلوب المتبعة في الفريق.

## الاختبار (Testing)

كتابة اختبارات الوحدة (**Unit Tests**) من المهام التي يؤديها **Claude** بكفاءة عالية، خصوصًا عند تزويده بالدالة المطلوب اختبارها وإطار الاختبار المستخدم في المشروع (مثل **Jest** أو **pytest**). يُنصح بمراجعة الاختبارات المولدة للتأكد من أنها تغطي الحالات الحدية فعليًا وليس فقط المسار السعيد.

## التوثيق (Documentation)

توليد التوثيق التقني — تعليقات الكود، ملفات **README**، أو توثيق **API** — من الاستخدامات العملية الشائعة، خصوصًا عند وجود كود بدون توثيق كافٍ.

## التعامل مع مشاريع كاملة

عند العمل مع مشاريع كبيرة تتجاوز نافذة سياق واحدة، يُفضّل تزويد **Claude** بملخص لبنية المشروع والملفات ذات الصلة فقط بدلاً من محاولة تحميل المستودع بأكمله. الفصل السابع يشرح كيف تتولى **Claude Code** هذه المهمة تلقائيًا عبر فهم بنية المستودع مباشرة.

## الفصل 7 - Claude Code

### ما هو Claude Code؟

**Claude Code** أداة سطر أوامر (CLI) تعمل داخل الطرفية (Terminal) وتتيح لـ **Claude** التفاعل المباشر مع مستودع الكود: قراءة الملفات، تعديلها، تشغيل الأوامر، واستخدام **Git**، ضمن صلاحيات يتحكم بها المستخدم. بخلاف واجهة المحادثة، يعمل **Claude Code** داخل بيئة المشروع الفعلية ويمكنه فهم بنية الكود بالكامل دون الحاجة لنسخ ولصق يدوي.

### التثبيت (Installation)

الطريقة الموصى بها من **Anthropic** هي المثبت المستقل (Native Installer)، والذي لا يتطلب **:Node.js**

```
# macOS / Linux / WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex
```

لا يزال التثبيت عبر **npm** مدعومًا كخيار بديل، ويتطلب **Node.js** بإصدار حديث:

```
npm install -g @anthropic-ai/claude-code
```

♦ **تجنب استخدام sudo مع أمر npm install؛ فهذا سبب شائع لمشاكل الصلاحيات لاحقًا. لأن طرق التثبيت وأرقام الإصدارات تتغير باستمرار، تحقق دائمًا من الخطوات المحدثة في الوثائق الرسمية قبل التثبيت.**

➔ [توثيق تثبيت Claude Code الرسمي](https://docs.claude.com/en/docs/claude-code/overview)

<https://docs.claude.com/en/docs/claude-code/overview>

الإعداد الأولي (Setup)

```

claude --version    # confirm installation
claude doctor      # full install & config health check
claude auth login   # log in and link your account
    
```

بعد تسجيل الدخول، ينتقل المستخدم إلى مجلد المشروع ويشغل الأمر **claude** لبدء جلسة تفاعلية داخل ذلك المشروع تحديدًا.

```

cd my-project
claude
    
```

## أهم الأوامر داخل الجلسة التفاعلية

| الوظيفة                                                 | الأمر                     |
|---------------------------------------------------------|---------------------------|
| عرض قائمة الأوامر المتاحة داخل الجلسة                   | <code>/help</code>        |
| إنشاء ملف <code>CLAUDE.md</code> يحتوي على سياق المشروع | <code>/init</code>        |
| مسح سياق المحادثة الحالية وبدء سياق جديد                | <code>/clear</code>       |
| تلخيص المحادثة الحالية لتقليل استهلاك نافذة السياق      | <code>/compact</code>     |
| تبديل النموذج المستخدم في الجلسة الحالية                | <code>/model</code>       |
| إدارة صلاحيات الوصول للملفات والأوامر                   | <code>/permissions</code> |
| إدارة الوكلاء الفرعيين (Subagents) المتاحين             | <code>/agents</code>      |

## أهم أوامر سطر النظام (خارج الجلسة)

```

claude update          # update Claude Code to the latest version
claude config         # view and edit settings
claude mcp add <name>  # add an MCP server to the project
    
```

## ملف `CLAUDE.md`

ملف `CLAUDE.md` هو ملف نصي يوضع في جذر المشروع ويحتوي على تعليمات ثابتة يقرأها Claude Code تلقائيًا في بداية كل جلسة: معايير الأسلوب البرمجي، أوامر التشغيل والاختبار، وأي قواعد خاصة بالمشروع. يُفضل إبقاؤه محدثًا كـ 'وثيقة حية' يُضاف إليها كلما لوحظ سلوك غير مرغوب يتكرر.

## سير عمل المشروع وفهم بنية الكود

عند بدء جلسة داخل مستودع، يستطيع Claude Code استكشاف بنية الملفات والاعتماديات (Dependencies) عند الحاجة، بدلاً من افتراض أن كل الكود قد قُري مسبقاً. يُنصح بمهام محددة النطاق (مثل 'أضف Endpoint جديد لتسجيل الدخول') بدلاً من مهام مبهمّة واسعة.

## سير عمل Git

يمكن لـ Claude Code تنفيذ عمليات Git مثل إنشاء فروع (Branches)، كتابة رسائل Commit، ومراجعة الفروقات (Diffs) قبل الدمج، ضمن الصلاحيات الممنوحة له. يبقى القرار النهائي بالدمج (Merge) أو الدفع (Push) إلى فرع رئيسي قراراً بشرياً في معظم بيئات العمل الاحترافية.

## الاختبار والأتمتة

يمكن دمج Claude Code ضمن سكربتات آلية أو خطوط أنابيب CI/CD عبر وضعه غير التفاعلي (Headless Mode)، لتنفيذ مهام محددة مسبقاً دون تدخل بشري مباشر، مثل مراجعة Pull Request تلقائياً أو توليد تقرير عن التغييرات.

## أفضل الممارسات

- ابدأ بمهام صغيرة ومحددة النطاق قبل تكليف Claude Code بمهام كبيرة عبر عدة ملفات.
- راجع دائماً التعديلات المقترحة قبل قبولها، خصوصاً في الكود الحساس مثل المصادقة والدفع.
- احتفظ بملف CLAUDE.md محدثاً ليقفل الحاجة لتكرار نفس التعليمات.
- استخدم clear/ أو compact/ عند طول الجلسة لتفادي استهلاك نافذة السياق دون داعٍ.
- قيّد الصلاحيات (Permissions) في المشاريع الحساسة بدلاً من منح وصول كامل افتراضياً.

## ➔ [خريطة توثيق Claude Code الكاملة](https://docs.claude.com/en/docs/claude-code/overview)

<https://docs.claude.com/en/docs/claude-code/overview>

## الفصل 8 - Claude API

### أساسيات API

يوفر **Anthropic API** وصولاً برمجياً مباشراً إلى نماذج **Claude** عبر نقطة نهاية (Endpoint) واحدة رئيسية هي **v1/messages**، مما يتيح دمج **Claude** داخل أي تطبيق أو خدمة.

### مفاتيح API والمصادقة

يتم إنشاء مفتاح **API** من لوحة تحكم **Anthropic Console**، ويُرسَل هذا المفتاح ضمن ترويسة (Header) كل طلب. يجب عدم تضمين المفتاح مباشرة داخل كود العميل (Client-side code) أو رفعه إلى مستودع عام؛ هذه النقطة تُفصّل أكثر في الفصل الثامن عشر.

### [Anthropic Console — إدارة مفاتيح API](https://console.anthropic.com) ↗

<https://console.anthropic.com>

### بنية الطلب والاستجابة

```
curl https://api.anthropic.com/v1/messages \
-H "x-api-key: $ANTHROPIC_API_KEY" \
-H "anthropic-version: 2023-06-01" \
-H "content-type: application/json" \
-d '{
  "model": "claude-sonnet-5",
  "max_tokens": 1024,
  "messages": [
```

```
{
  "role": "user",
  "content": "Explain what a race condition is."
}
```

الاستجابة تعود بصيغة JSON تحتوي على حقل **content**، وهو مصفوفة من الكتل (Blocks) قد تشمل نصًا أو استدعاء أداة (Tool Use) بحسب الطلب.

## أهم المعاملات (Parameters)

| المعامل     | الوظيفة                                              |
|-------------|------------------------------------------------------|
| model       | اسم النموذج المستخدم، مثل <b>claude-sonnet-5</b>     |
| max_tokens  | الحد الأقصى لعدد Tokens في الرد                      |
| messages    | سجل المحادثة بين المستخدم والنموذج                   |
| system      | تعليمات النظام التي تحدد سلوك النموذج العام          |
| temperature | درجة العشوائية في الاستجابة                          |
| stream      | تفعيل استقبال الاستجابة تدريجيًا بدلاً من دفعة واحدة |

## تعليمات النظام (System Prompts)

حقل **system** منفصل عن حقل **messages**، ويُستخدم لتحديد دور النموذج وسلوكه العام طوال المحادثة، مثل تحديد نبرة الرد أو تقييد النموذج بمجال معين من المعرفة.



## البث (Streaming)

عند تفعيل **stream: true**، تصل الاستجابة على شكل أحداث متتالية (Server-Sent Events) بدلاً من انتظار اكتمال الرد كاملاً، مما يحسّن تجربة المستخدم في الواجهات التفاعلية حيث يظهر النص تدريجياً أثناء توليده.

## معالجة الأخطاء

تعيد الـ API رموز حالة HTTP قياسية للإشارة إلى نوع الخطأ: 401 لمشاكل المصادقة، 429 عند تجاوز حدود الاستخدام (Rate Limits)، و 500 لأخطاء الخادم. يُنصح ببناء منطق إعادة المحاولة (Retry) مع تأخير تصاعدي (Exponential Backoff) خصوصاً عند التعامل مع الخطأ 429.

## حدود الاستخدام (Rate Limits)

تُفرض حدود على عدد الطلبات وعدد Tokens في الدقيقة، وتختلف هذه الحدود حسب مستوى الحساب وخطة الاستخدام. عند بناء تطبيق إنتاجي بحجم طلبات كبير، يجب تصميم النظام مع افتراض وجود هذه الحدود منذ البداية.

➔ [توثيق الـ API الرسمي الكامل](https://docs.claude.com/en/api/overview)

<https://docs.claude.com/en/api/overview>

## الفصل 9 - استخدام الـ API عمليًا

### Python

توفّر Anthropic حزمة SDK رسمية للغة Python تبسّط التعامل مع الـ API دون الحاجة لبناء طلبات HTTP يدويًا:

```
pip install anthropic

import anthropic

client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY from
the environment

message = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "Summarize this text in two
sentences: ..."}
    ],
)

print(message.content[0].text)
```

## JavaScript / TypeScript

```

npm install @anthropic-ai/sdk

import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

const message = await client.messages.create({
  model: "claude-sonnet-5",
  max_tokens: 1024,
  messages: [{ role: "user", content: "Write a haiku about
debugging." }],
});

console.log(message.content[0].text);

```

## REST مباشر مقابل SDK

استخدام SDK الرسمي مفضل في معظم الحالات لأنه يتولى تفاصيل مثل إعادة المحاولة، وتحليل الاستجابة، ودعم أنواع بيانات TypeScript. الاتصال المباشر بواجهة REST مفيد عند العمل بلغة لا تملك SDK رسميًا، أو عند الحاجة للتحكم الكامل في الطلب.

## بناء تطبيق بسيط

مثال توضيحي: أداة سطر أوامر تلخص ملف نصي يمرره المستخدم. الخطوات الأساسية: قراءة الملف، إرساله ضمن رسالة المستخدم مع تعليمات تلخيص واضحة، ثم طباعة الرد. هذا النمط — قراءة مدخل، صياغة أمر، استدعاء الـ API، معالجة المخرج — هو الأساس الذي تُبنى عليه معظم تطبيقات Claude API الحقيقية بغض النظر عن حجمها.

## التعامل مع الملفات

يمكن إرسال الصور ومستندات PDF إلى الـ API كبيانات Base64 ضمن حقل `content`، مع تحديد `media_type` المناسب. هذا يتيح بناء تطبيقات تحلل مستندات أو صورًا برمجياً دون تدخل بشري في كل طلب.

## المخرجات المهيكلة (Structured Outputs)

عند الحاجة لنتائج بصيغة قابلة للمعالجة برمجياً مثل JSON، يُفضّل تحديد الصيغة المطلوبة بدقة ضمن تعليمات النظام، وطلب تعريف Tool مخصص لهذا الغرض عند الحاجة لضمان اتساق البنية، كما هو موضح في الفصل العاشر حول Tool Use.

## اعتبارات الإنتاج (Production Considerations)

- لا تُخزن مفاتيح الـ API داخل الكود؛ استخدم متغيرات بيئة أو خدمة إدارة أسرار ( Secrets Manager ).
- صمم منطق إعادة المحاولة مع التعامل مع حدود الاستخدام.
- راقب تكلفة الاستخدام عبر Anthropic Console بشكل دوري.
- استخدم التخزين المؤقت للسياق (Prompt Caching) عند تكرار نفس التعليمات الطويلة في طلبات متعددة لتقليل التكلفة.

## الفصل 10 - استخدام الأدوات (Tool Use)

### ما هو Tool Use؟

**Tool Use** (المعروف أيضاً باسم **Function Calling**) هو آلية تتيح لنموذج Claude طلب تنفيذ دالة أو أداة خارجية أثناء المحادثة، بدلاً من الاكتفاء بتوليد نص. النموذج نفسه لا ينفذ الأداة؛ بل يطلب من التطبيق المستضيف تنفيذها، ثم يستقبل النتيجة ليكمل الرد بناءً عليها.

### كيف تعمل الآلية؟

تمر العملية بأربع خطوات: تعريف الأدوات المتاحة ضمن الطلب، قرار النموذج باستدعاء أداة معينة مع مدخلات محددة، تنفيذ الأداة فعلياً من طرف التطبيق، ثم إعادة إرسال نتيجة التنفيذ إلى النموذج ليولد الرد النهائي.

### تعريف الأدوات (Tool Definitions)

```
{
  "name": "get_weather",
  "description": "Get the current weather for a given city.",
  "input_schema": {
    "type": "object",
    "properties": {
      "city": { "type": "string" }
    },
    "required": ["city"]
  }
}
```

```
}
}
```

دقة الوصف (description) وحقول input\_schema تؤثر بشكل مباشر على مدى صحة استخدام النموذج للأداة؛ وصف غامض يزيد احتمال استدعاء الأداة في غير محلها أو بمدخلات غير صحيحة.

### استدعاءات الأدوات (Tool Calls)

عندما يقرر النموذج استخدام أداة، تحتوي الاستجابة على كتلة من نوع tool\_use تتضمن اسم الأداة والمدخلات المقترحة. يقوم التطبيق بتنفيذ الأداة فعليًا، ثم يرسل النتيجة ضمن رسالة جديدة من نوع tool\_result.

### مثال عملي مبسط

تطبيق مساعد حجوزات يمتلك أداتين: check\_availability و book\_room. عندما يطلب المستخدم حجز غرفة في تاريخ معين، يستدعي النموذج أولاً check\_availability للتأكد من التوفر، ثم بناءً على النتيجة يقرر استدعاء book\_room أو إبلاغ المستخدم بعدم التوفر. هذا التسلسل يحدث تلقائيًا ضمن حلقة الطلب والاستجابة دون الحاجة لكتابة منطق تفرّع يدوي معقد.

### متى يُستخدم Tool Use؟

- عندما يحتاج التطبيق لبيانات لحظية لا يملكها النموذج، مثل حالة الطقس أو رصيد حساب.
- عندما يحتاج التطبيق لتنفيذ إجراء فعلي، مثل إرسال بريد إلكتروني أو تحديث قاعدة بيانات.
- عندما يكون المطلوب ناتجًا بصيغة مهيكلة صارمة يمكن ضمانها عبر Tool Schema.

## الفصل 11 - بروتوكول (MCP) Model Context Protocol

### ما هو MCP؟

**Model Context Protocol** هو بروتوكول مفتوح طوّره **Anthropic** لتوحيد طريقة اتصال نماذج اللغة بمصادر البيانات والأدوات الخارجية، مثل أنظمة الملفات، قواعد البيانات، وواجهات برمجية لخدمات مثل **Slack** أو **Google Drive** أو **Asana**.

### لماذا تم تطويره؟

قبل **MCP**، كان دمج نموذج لغوي مع كل خدمة خارجية يتطلب بناء تكامل مخصص منفصل. يوفر **MCP** طبقة موحدة: بمجرد بناء خادم **MCP** لخدمة معينة مرة واحدة، يمكن لأي تطبيق يدعم البروتوكول استخدامه دون إعادة بناء التكامل من الصفر لكل عميل.

## البنية العامة (Architecture)

يقوم MCP على نموذج عميل - خادم (Client-Server):

| الدور                                                                                   | المكون     |
|-----------------------------------------------------------------------------------------|------------|
| يعرض أدوات وموارد وقوالب أوامر مرتبطة بخدمة أو نظام معين                                | MCP Server |
| التطبيق المستضيف (مثل Claude Code أو Claude Desktop) الذي يتصل بالخادم ويستخدم ما يعرضه | MCP Client |
| إجراءات قابلة للاستدعاء، مثل إنشاء مهمة أو إرسال رسالة                                  | Tools      |
| بيانات يمكن قراءتها، مثل محتوى ملف أو نتيجة استعلام                                     | Resources  |
| قوالب أوامر جاهزة يوفرها الخادم لتسهيل استخدام معين                                     | Prompts    |

## كيفية استخدام MCP

يمكن إضافة خادم MCP إلى Claude Code عبر سطر الأوامر، أو ضمن إعدادات Claude Desktop، أو استدعاؤه برمجياً عبر الـ API باستخدام معامل `mcp_servers` كما هو موضح في الفصل التاسع.

```
claude mcp add my-server --command "node server.js"
```

## أمثلة وحالات استخدام حقيقية

- ربط Claude Code بنظام تتبّع المهام (مثل Jira أو Asana) لقراءة المهام المفتوحة وتحديث حالتها.



- ربط Claude بقاعدة بيانات داخلية للاستعلام عن بيانات مبيعات وتلخيصها دون كتابة استعلامات SQL يدويًا.
- بناء خادم MCP مخصص يعرض وثائق الشركة الداخلية كموارد قابلة للبحث.

◆ بناء خادم MCP مخصص يتطلب معرفة برمجية أساسية (عادة Python أو TypeScript)، لكن استخدام خوادم MCP جاهزة من مزودي خدمات معروفين لا يتطلب أكثر من ربط الحساب.

➔ [توثيق Model Context Protocol الرسمي](https://docs.claude.com/en/docs/agents-and-tools/mcp)

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

## الفصل 12 - وكلاء Claude والعمل الوكيل (Agentic Workflows)

### ما هو AI Agent؟

الوكيل (Agent) هو نظام يستخدم نموذجًا لغويًا لاتخاذ قرارات متتالية وتنفيذ إجراءات فعلية عبر أدوات متاحة له، بهدف إنجاز مهمة كاملة دون الحاجة لتوجيه بشري في كل خطوة. الفارق الجوهرى بين الوكيل والردشة العادية هو أن الوكيل يخطط وينفذ ويقيم النتائج، بينما الدردشة تنتج ردًا واحدًا لكل رسالة.

### Agent مقابل Chatbot

| الخاصية         | Chatbot                       | Agent                               |
|-----------------|-------------------------------|-------------------------------------|
| طبيعة التفاعل   | سؤال وجواب مباشر              | تخطيط وتنفيذ متعدد الخطوات          |
| استخدام الأدوات | محدود أو غير موجود            | أساسي لإنجاز المهمة                 |
| اتخاذ القرار    | يعتمد على المستخدم في كل خطوة | يتخذ قرارات فرعية ضمن حدود محددة    |
| مثال            | الإجابة عن سؤال تقني          | إصلاح أخطاء في مستودع كامل تلقائيًا |

### حلقة عمل الوكيل (Agent Loop)

تعمل معظم الوكلاء وفق حلقة متكررة: الملاحظة (فهم الحالة الراهنة)، التخطيط (تحديد الخطوة التالية)، التنفيذ (استدعاء أداة)، ثم التقييم (مراجعة النتيجة قبل الانتقال للخطوة التالية). تستمر هذه الحلقة حتى تحقيق الهدف أو الوصول لحد أقصى من الخطوات.

## التخطيط والذاكرة

في المهام الطويلة، يحتاج الوكيل للاحتفاظ بسياق ما تم إنجازه سابقًا دون إعادة تحميل كل التفاصيل في كل خطوة. تُحل هذه المشكلة عادة عبر تلخيص الخطوات السابقة دوريًا، أو تخزين حالة وسيطة خارج نافذة السياق مباشرة (مثل ملف أو قاعدة بيانات) والرجوع إليها عند الحاجة.

## بناء وكيل بسيط

مثال: وكيل لمراجعة طلبات الدمج (Pull Requests) تلقائيًا. مكوناته الأساسية: أداة لقراءة الفروقات (Diff)، أداة لتشغيل الاختبارات، وأداة لإضافة تعليق على الطلب. تعليمات النظام تحدد للوكيل: اقرأ الفروقات، شغل الاختبارات، إذا فشلت أضف تعليقًا يوضح السبب، وإذا نجحت أضف تعليق موافقة أولية. هذا النمط البسيط يوضح الفكرة الأساسية التي تُبنى عليها معظم الوكلاء الإنتاجية.

## أفضل الممارسات

- حدد نطاق صلاحيات الوكيل بوضوح؛ لا تمنحه صلاحيات أوسع من اللازم لإنجاز مهمته.
- ضع حدًا أقصى لعدد الخطوات أو الوقت لتفادي حلقات تنفيذ غير منتهية.
- أضف نقاط تحقق بشرية (Human-in-the-loop) في القرارات عالية المخاطر مثل الدفع المالي أو حذف بيانات.
- سجل خطوات الوكيل لتسهيل تتبع الأخطاء ومراجعة القرارات لاحقًا.

## الفصل 13 - Claude في هندسة البرمجيات

### تصميم البنية البرمجية (Software Architecture)

يمكن الاستعانة بـ Claude لمناقشة خيارات معمارية — مثل Monolith مقابل Microservices، أو اختيار نمط تخزين بيانات مناسب — بتزويده بمتطلبات المشروع الفعلية: حجم البيانات المتوقع، عدد المستخدمين، وقيود الفريق. النقاش المعماري المفيد يتطلب سياقاً حقيقياً، وليس أسئلة نظرية عامة.

#### تحليل المتطلبات

يمكن استخدام Claude لتحويل وصف عمل غير منظم (مثل ملاحظات اجتماع) إلى متطلبات وظيفية مرقمة وقابلة للتتبع، وهي خطوة مفيدة قبل بدء التصميم التقني الفعلي.

#### UML وتصميم قواعد البيانات

يمكن لـ Claude اقتراح مخططات UML نصية (مثل صيغة Mermaid) لتوضيح تدفق العمليات أو العلاقات بين الكيانات، كما يمكنه اقتراح مخطط قاعدة بيانات علائقية انطلاقاً من وصف الكيانات وعلاقاتها.

### Mobile و Frontend و Backend

يغطي Claude تطوير الواجهات الخلفية (بناء APIs، التعامل مع قواعد البيانات، المصادقة)، والواجهات الأمامية (React و Vue وما شابه)، وتطوير الجوال (Swift و Kotlin و Flutter). في المشاريع متعددة الطبقات، يُفضّل التعامل مع كل طبقة في سياق منفصل نسبياً بدلاً من خلط تفاصيل الواجهة الخلفية والأمامية في نفس الطلب.

## الاختبار وCI/CD

بالإضافة إلى كتابة اختبارات الوحدة، يمكن لـ **Claude** المساعدة في كتابة ملفات إعداد خطوط أنابيب **CI/CD** (مثل **GitHub Actions**) وتشخيص أسباب فشل عمليات البناء (**Build Failures**) عند تزويده بسجلات الخطأ (**Logs**) كاملة.

## مراجعة الكود وسير عمل GitHub

يمكن دمج **Claude** ضمن سير عمل مراجعة الكود على **GitHub**، سواء عبر **Claude Code** محليًا قبل فتح طلب الدمج، أو عبر أتمتة تستدعي الـ **API** لمراجعة كل **Pull Request** تلقائيًا وإضافة ملاحظات أولية قبل المراجعة البشرية.

## التوثيق التقني للمشاريع

توثيق قرارات التصميم (**Architecture Decision Records**)، وتوثيق الـ **API** الداخلي، وأدلة الإعداد (**Onboarding Guides**) من المهام التي توفر وقتًا كبيرًا عند الاستعانة بـ **Claude**، خصوصًا عند تزويده بالكود الفعلي بدلاً من وصف عام للمشروع.

## الفصل 14 - Claude للطلاب والتعلم

### الدراسة باستخدام Claude

الاستخدام الفعال لـ Claude في الدراسة لا يعني طلب الإجابات النهائية مباشرة، بل استخدامه كأداة لفهم المفاهيم وبناء المعرفة تدريجياً. طلب شرح مفهوم مع طلب أسئلة اختبارية للتأكد من الفهم أكثر فائدة على المدى الطويل من طلب حل الواجب مباشرة.

### فهم المحاضرات وتلخيص الكتب

يمكن رفع نص محاضرة أو فصل من كتاب وطلب تلخيص النقاط الرئيسية، أو طلب شرح فقرة معقدة بلغة أبسط. يُفضل طلب التلخيص بصيغة تحافظ على الترتيب المنطقي للأفكار بدلاً من قائمة نقاط مبتورة عن سياقها.

### حل المسائل

في المواد التقنية والرياضية، الأسلوب الأكثر فائدة هو طلب شرح خطوات الحل مع التبرير المنطقي لكل خطوة، ثم محاولة حل مسألة مشابهة بشكل مستقل للتأكد من استيعاب الطريقة وليس فقط الإجابة.

### تعلم البرمجة

بالنسبة لمتعلمي البرمجة، يُنصح بطلب شرح الكود الذي يكتبه Claude بدلاً من نسخه مباشرة دون فهم، وطلب تمارين تدريجية الصعوبة على نفس المفهوم لترسيخه.

### إنشاء خطة دراسة

يمكن لـ Claude اقتراح خطة دراسة مقسّمة على أسابيع بناءً على المادة والوقت المتاح ومستوى المتعلم الحالي، مع نقاط مراجعة دورية للتأكد من التقدم الفعلي.

## الاستعداد للمقابلات التقنية

يمكن استخدام Claude لمحاكاة مقابلة تقنية عبر طرح أسئلة تدريجية الصعوبة وتقديم ملاحظات على الحلول المقترحة، بالإضافة إلى مراجعة سيرة ذاتية أو مشروع شخصي قبل تقديمه.

## الفصل 15 - Claude للإنتاجية

### الكتابة

من كتابة المسودات الأولى إلى تحرير نصوص موجودة، يعمل Claude بشكل أفضل عندما يُزود بجمهور مستهدف واضح ونبرة محددة، بدلاً من طلب عام مثل "اكتب نصًا جيدًا".

### البحث والتلخيص

يمكن استخدام Claude لتلخيص مستندات طويلة أو مقارنة عدة مصادر حول موضوع واحد، مع ضرورة التحقق من أي معلومة حساسة أو حديثة من مصدرها الأصلي بدلاً من الاعتماد الكامل على التلخيص.

### تحليل البيانات

عند رفع جداول بيانات أو ملفات CSV، يمكن لـClaude اقتراح تحليلات واستنتاجات أولية، وهو مفيد بشكل خاص في مرحلة الاستكشاف الأولى للبيانات قبل بناء تحليل إحصائي رسمي.

### البريد الإلكتروني والتقارير

صياغة رسائل بريد إلكتروني حساسة (مثل رفض عرض عمل أو التعامل مع شكوى) أو تقارير دورية تستفيد بشكل خاص من تزويد Claude بالسياق الكامل للموقف، بما في ذلك العلاقة مع الطرف الآخر والهدف المطلوب من الرسالة.

### العصف الذهني والتخطيط

طرح فكرة أولية وطلب توسيعها إلى عدة اتجاهات ممكنة، أو طلب نقد بناء لخطّة موجودة، من الاستخدامات الفعالة لـClaude في مراحل التخطيط المبكرة.



## الآتمة

عبر الـ **API** أو **MCP**، يمكن دمج **Claude** في مهام إنتاجية متكررة مثل تصنيف الرسائل الواردة، أو توليد ملخصات أسبوعية تلقائية من بيانات متعددة المصادر، كما هو موضح في الفصلين التاسع والحادي عشر.

## الفصل 16 - Claude في المحتوى والعمل الإبداعي

### الكتابة الإبداعية

يمكن لـ **Claude** المساعدة في كتابة القصص والسيناريوهات، لكن النتائج الأفضل تأتي من التعامل معه كشريك في العصف الذهني وتطوير الأفكار، وليس كمصدر وحيد للنص النهائي. تحديد الأسلوب والصوت السردية المطلوب منذ البداية يحسّن جودة الناتج بشكل كبير.

### إنشاء المحتوى ومنصات التواصل

عند طلب محتوى لمنصة معينة (مثل LinkedIn أو منشور مدونة)، يُفضّل تحديد طول المحتوى ونبرته والجمهور المستهدف بدلاً من طلب "منشور جيد" بشكل عام.

### التسويق

كتابة نصوص إعلانية، وصف منتجات، أو رسائل بريد تسويقية من الاستخدامات الشائعة، مع التنبيه إلى ضرورة مراجعة أي ادعاءات تسويقية دقيقة (أرقام، مقارنات) من مصدرها قبل النشر.

### التحرير والمراجعة

مراجعة نص موجود لتحسين الوضوح أو تصحيح الأخطاء اللغوية من المهام التي يؤديها **Claude** بفعالية عالية، خصوصاً عند تحديد نوع المراجعة المطلوبة: لغوية فقط، أم أسلوبية، أم هيكلية.

### سير العمل الإبداعي

في المشاريع الإبداعية الطويلة، يفيد تقسيم العمل إلى مراحل: التخطيط والهيكلية أولاً، ثم الكتابة الفعلية، ثم المراجعة، بدلاً من طلب العمل كاملاً دفعة واحدة، لأن كل مرحلة تستفيد من أوامر وسياق مختلف.

## الفصل 17 - سير عمل متقدمة مع Claude

### المهام متعددة الخطوات

المهام المعقدة تُنجز بجودة أعلى عند تقسيمها إلى سلسلة خطوات واضحة بدلاً من طلب واحد ضخم. على سبيل المثال، بدلاً من طلب 'حلّ هذا التقرير المالي واكتب توصيات'، يُفضّل: استخراج الأرقام الرئيسية أولاً، ثم تحليل الاتجاهات، ثم صياغة التوصيات بناءً على التحليل السابق.

### سلسلة الأوامر (Prompt Chaining)

سلسلة الأوامر تعني تمرير مخرجات خطوة كمدخلات للخطوة التالية، مما يتيح مراجعة النتائج الوسيطة وتصحيح المسار قبل الوصول للمخرج النهائي، وهو أكثر موثوقية من محاولة إنجاز كل شيء في طلب واحد معقد.

### Claude مع الأدوات الخارجية والـ APIs

دمج Claude مع أنظمة خارجية عبر Tool Use أو MCP يفتح إمكانيات بناء سير عمل يتجاوز حدود المحادثة النصية، مثل قراءة بيانات حيّة من نظام داخلي، اتخاذ قرار بناءً عليها، ثم تحديث نظام آخر بالنتيجة، دون تدخل بشري في كل خطوة.

### Claude و GitHub

ربط Claude Code أو الـ API بسير عمل GitHub (عبر GitHub Actions أو Webhooks) يتيح أتمتة مهام مثل مراجعة كل طلب دمج جديد تلقائياً، أو تصنيف المشكلات (Issues) الواردة حسب الأولوية والنوع.

## أمثلة سير عمل شامل

مثال: خط أنابيب لمعالجة تذاكر الدعم الفني. الخطوة الأولى: تصنيف التذكرة (فئة، أولوية) باستخدام **Haiku** لسرعته وانخفاض تكلفته نظرًا لحجم التذاكر الكبير. الخطوة الثانية: لتذاكر الأولوية العالية فقط، استخدام **Sonnet** لصياغة رد أولي مفصل يراجع موظف الدعم قبل الإرسال. هذا النمط — استخدام نموذج أخف للفرز ونموذج أقوى للمهام الحساسة — يوازن بين التكلفة والجودة في الأنظمة كبيرة الحجم.

## الفصل 18 - الأمان والخصوصية والاستخدام المسؤول

### الخصوصية والتعامل مع البيانات

عند إرسال بيانات حساسة إلى Claude — سواء عبر واجهة المحادثة أو الـ API — يجب معرفة سياسة الاحتفاظ بالبيانات المطبقة على نوع الحساب المستخدم، لأنها تختلف بين الحسابات الفردية وحسابات الأعمال (Enterprise). يُنصح دائماً بمراجعة سياسة الخصوصية الرسمية بدلاً من الافتراض.

➔ [مركز الثقة والخصوصية لدى Anthropic](https://trust.anthropic.com)

<https://trust.anthropic.com>

### أمان مفاتيح الـ API

- لا تُضمّن مفاتيح الـ API مباشرة في كود يعمل على المتصفح (Client-side)؛ استخدم خادماً وسيطاً (Backend) دائماً.
- لا ترفع المفاتيح إلى مستودعات Git عامة؛ استخدم متغيرات بيئة أو خدمة إدارة أسرار.
- أنشئ مفاتيح منفصلة لكل بيئة (تطوير، اختبار، إنتاج) لتسهيل إبطال مفتاح واحد دون التأثير على البقية.

### حقن الأوامر (Prompt Injection)

حقن الأوامر هو محاولة تضمين تعليمات ضارة داخل بيانات يعالجها النموذج (مثل محتوى صفحة ويب أو مستند مرفوع) بهدف التلاعب بسلوكه. عند بناء تطبيقات تعالج محتوى من مصادر خارجية غير موثوقة، يجب التعامل مع ذلك المحتوى كبيانات فقط، وليس كتعليمات، وفصل تعليمات النظام بوضوح عن أي محتوى خارجي يُمرّر للنموذج.

## المعلومات الحساسة

يجب تجنب إرسال معلومات تعريف شخصية حساسة أو بيانات سرية للشركة إلى أي خدمة سحابية، بما فيها Claude، دون التحقق من أن ذلك يتوافق مع سياسات المؤسسة ومتطلبات الامتثال المعمول بها.

## أفضل ممارسات الأمان

- طبق مبدأ أقل الصلاحيات (Least Privilege) عند منح Claude Code أو الوكلاء صلاحيات وصول لأنظمة حقيقية.
- راجع أي كود أو إجراء يقترحه النموذج قبل تنفيذه في بيئة إنتاجية.
- سجل (Log) استدعاءات الأدوات في الأنظمة الوكيلية لتسهيل التدقيق لاحقًا.

## الاستخدام المسؤول للذكاء الاصطناعي

الاستخدام المسؤول يعني إدراك حدود النموذج: قد يخطئ، قد يقدم معلومات قديمة أو غير دقيقة، ولا ينبغي الاعتماد عليه كمصدر وحيد في قرارات طبية أو قانونية أو مالية حساسة دون تحقق بشري ومهني مختص.

## الفصل 19 - مشاريع تطبيقية واقعية

### مشروع 1 — مساعد برمجي داخل الطرفية

الهدف: أداة CLI تستخدم Claude Code لمراجعة كل Commit جديد محليًا قبل الدفع (Push).

- أضف ملف `CLAUDE.md` يوضح معايير الكود المتبعة في المشروع.
- أنشئ Hook محلي (Git pre-push hook) يستدعي `claude` بوضع غير تفاعلي لمراجعة الفروقات.
- اطلب من Claude تلخيص المخاطر المحتملة في التعديل قبل السماح بالدفع.

### مشروع 2 — محلّ مستندات

الهدف: تطبيق ويب بسيط يستقبل ملف PDF ويستخرج منه بيانات محددة (مثل الفواتير: التاريخ، المبلغ، المورد).

- استخدم الـ API لإرسال المستند كـ Base64 مع تعليمات استخراج دقيقة.
- عرّف Tool مخصص (`extract_invoice_data`) لضمان ناتج JSON منظم.
- خزّن النتائج في قاعدة بيانات لتحليلها لاحقًا.

### مشروع 3 — مساعد دراسي تفاعلي

الهدف: تطبيق يحوّل مادة دراسية مرفوعة إلى بطاقات مراجعة (Flashcards) وأسئلة اختبارية تدريجية الصعوبة.

- قسّم المحتوى إلى وحدات مواضيعية صغيرة قبل إرساله للنموذج.

- اطلب توليد أسئلة بمستويات صعوبة مختلفة (تذكر، فهم، تطبيق).
- احتفظ بسجل أداء المستخدم لتحديد المواضيع التي تحتاج مراجعة إضافية.

#### مشروع 4 — تطبيق مبني على الـ API بالكامل

الهدف: خدمة تصنيف تذاكر دعم فني تلقائيًا حسب الفئة والأولوية باستخدام Haiku للسرعة، مع تصعيد الحالات الحرجة إلى Sonnet لصياغة رد أولي.

#### مشروع 5 — تكامل عبر MCP

الهدف: ربط Claude Code بخادم MCP مخصص يتيح الاستعلام عن حالة الخوادم الداخلية للشركة، بحيث يمكن لمهندس البنية التحتية سؤال Claude مباشرة عن حالة النظام دون فتح لوحة تحكم منفصلة.

#### مشروع 6 — وكيل أتمتة سير عمل

الهدف: وكيل يراقب صندوق بريد وارد، يصنّف الرسائل، يستخرج المهام المطلوبة منها تلقائيًا، وينشئها في نظام إدارة المهام عبر MCP، مع ترك المهام الحساسة (مثل الطلبات المالية) لمراجعة بشرية قبل التنفيذ.



## الفصل 20 خارطة طريق احترافية مع Claude

### كيف تصبح محترفاً في استخدام Claude؟

الاحتراف في التعامل مع Claude ليس معرفة نظرية بل نتاج ممارسة متراكمة: كتابة أوامر، ملاحظة أين تفشل النتائج، وتعديل الأسلوب بناءً على ذلك. المسار التالي مقترح للتقدم بشكل تدريجي ومنظم.

### مسار التعلم المقترح

| المرحلة   | التركيز                                                              |
|-----------|----------------------------------------------------------------------|
| المبتدئ   | واجهة المحادثة، أساسيات Prompt Engineering،<br>الفصول 5-1            |
| المتوسط   | Claude Code، الـ API الأساسي، Tool Use، الفصول<br>10-6               |
| المتقدم   | MCP، الوكلاء الذكيون، سير العمل المعقد، الفصول 13-11<br>و17          |
| الاحترافي | بناء أنظمة إنتاجية كاملة، الأمان، المشاريع الواقعية، الفصول<br>19-18 |

### تطوير المهارات عبر المشاريع

أفضل طريقة لترسيخ أي مفهوم من هذا الكتاب هي تطبيقه على مشروع شخصي حقيقي، ولو صغير الحجم، بدلاً من الاكتفاء بالقراءة. مشاريع الفصل التاسع عشر نقطة انطلاق جيدة يمكن البناء عليها وتوسيعها.

## مصادر التعلم المستمر

- الوثائق الرسمية: تُحدَّث باستمرار مع كل إصدار جديد وتبقى المصدر الأدق.
- سجل التغييرات (Changelog) الخاص بـ Claude Code والـ API لمتابعة الميزات الجديدة.
- إعلانات Anthropic الرسمية لمتابعة النماذج والأدوات الجديدة فور صدورها.

➔ [مدونة الإعلانات الرسمية لـ Anthropic](https://www.anthropic.com/news)

<https://www.anthropic.com/news>

## بناء معرض أعمال (Portfolio)

توثيق المشاريع التي تستخدم Claude API أو MCP أو الوكلاء الذكية في معرض أعمال شخصي (مثل GitHub) يعزز الفرص المهنية، خصوصًا عند توضيح المشكلة التي حلها كل مشروع وليس فقط الكود المستخدم.

## فرص العمل المرتبطة

الطلب يتزايد على أدوار مثل مهندس أنظمة الذكاء الاصطناعي التطبيقي (AI Engineer)، ومهندس الوكلاء الذكية (Agent Engineer)، ومتخصصي هندسة الأوامر (Prompt Engineer) في فرق المنتج. المهارة الأساسية المشتركة بين هذه الأدوار هي القدرة على دمج نماذج اللغة ضمن أنظمة برمجية حقيقية موثوقة، وليس فقط استخدامها كأداة محادثة.

## الخطوات التالية

الخطوة الأنسب بعد إنهاء هذا الكتاب هي اختيار مشروع واحد فعلي — ولو بسيطاً — وبناءه بالكامل باستخدام ما لا يقل عن ثلاث أدوات من هذا الكتاب معًا: مثلاً Claude Code لكتابة الكود، الـ API لتشغيله في الإنتاج، و MCP لربطه بنظام حقيقي.

## خاتمة

يغطي هذا الكتاب مسارًا كاملاً من التعرف على Claude كأداة محادثة إلى بناء أنظمة وكيلية معقدة تعتمد على الـ API و MCP. الفكرة المحورية التي تربط كل فصل بالآخر هي أن جودة النتائج مع Claude ترتبط بشكل مباشر بوضوح السياق والهدف المقدم له، سواء كان ذلك في أمر محادثة بسيط أو في تعريف أداة داخل نظام وكيل معقد.

أدوات الذكاء الاصطناعي التوليدي، بما فيها Claude، تتطور بسرعة تفوق سرعة تحديث أي مرجع مطبوع. لهذا صُمم هذا الكتاب ليكون أساساً مفاهيمياً متيناً يبقى صالحاً حتى مع تغير التفاصيل التقنية، مع الإشارة المستمرة إلى الوثائق الرسمية كمصدر للتفاصيل الدقيقة والمحدثة.

## المراجع والمصادر الرسمية

جميع المصادر التالية رسمية وصادرة عن Anthropic مباشرة:

### Anthropic والشركة

الموقع الرسمي ↗

<https://www.anthropic.com>

مدونة الإعلانات والأخبار ↗

<https://www.anthropic.com/news>

مركز الثقة والخصوصية ↗

<https://trust.anthropic.com>

### Claude.ai

مركز المساعدة ↗

<https://support.claude.com>

### Claude API

التوثيق الرسمي ↗

<https://docs.claude.com/en/api/overview>

خريطة التوثيق الكاملة ↗

[https://docs.claude.com/en/docs\\_site\\_map.md](https://docs.claude.com/en/docs_site_map.md)

لوحة التحكم (Console) ↗

<https://console.anthropic.com>

## Claude Code

↗ [التوثيق الرسمي](#)

<https://docs.claude.com/en/docs/claude-code/overview>

↗ [حزمة npm الرسمية](#)

<https://www.npmjs.com/package/@anthropic-ai/claude-code>

## Model Context Protocol

↗ [التوثيق الرسمي لـ MCP](#)

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

## النماذج

↗ [نظرة عامة على نماذج Claude](#)

<https://docs.claude.com/en/docs/about-claude/models/overview>

## قاموس المصطلحات (Glossary)

| المصطلح                                             | الشرح                                                                          |
|-----------------------------------------------------|--------------------------------------------------------------------------------|
| <b>AI</b> <b>Artificial Intelligence</b>            | الذكاء الاصطناعي؛ أنظمة قادرة على أداء مهام تتطلب عادةً ذكاءً بشرياً.          |
| <b>LLM</b> <b>Large Language Model</b>              | نموذج لغوي كبير مُدرَّب على كميات ضخمة من النصوص لتوليد وفهم اللغة الطبيعية.   |
| <b>Prompt</b>                                       | النص المُدخل الذي يوجّه استجابة النموذج.                                       |
| <b>Context</b>                                      | المعلومات المُقدّمة للنموذج ليبنى عليها إجابته.                                |
| <b>Token</b>                                        | وحدة قياس النص الأساسية التي يتعامل بها النموذج.                               |
| <b>Context Window</b>                               | الحد الأقصى من النص الذي يمكن للنموذج معالجته في طلب واحد.                     |
| <b>Agent</b>                                        | نظام قائم على نموذج لغوي قادر على اتخاذ إجراءات متعددة الخطوات بشكل شبه مستقل. |
| <b>Tool</b>                                         | قدرة خارجية (مثل البحث أو تنفيذ كود) يمكن للنموذج استدعاؤها أثناء العمل.       |
| <b>API</b> <b>Application Programming Interface</b> | واجهة برمجية تتيح للتطبيقات التواصل مع خدمة أو نظام آخر.                       |

| المصطلح                            | الشرح                                                                  |
|------------------------------------|------------------------------------------------------------------------|
| CLI Command Line Interface         | واجهة سطر أوامر للتفاعل مع برنامج عبر نصوص مكتوبة.                     |
| Repository                         | مستودع يحتوي على كود المشروع وتاريخ تغييراته (عادة عبر Git).           |
| Refactoring                        | إعادة هيكلة الكود لتحسين جودته دون تغيير سلوكه الوظيفي.                |
| Debugging                          | عملية تحديد وإصلاح الأخطاء البرمجية.                                   |
| Code Review                        | مراجعة الكود من قبل شخص أو أداة أخرى قبل دمجها في المشروع.             |
| Hallucination                      | توليد النموذج لمعلومة تبدو واثقة لكنها غير صحيحة أو غير موجودة فعليًا. |
| RAG Retrieval-Augmented Generation | أسلوب يجمع بين استرجاع معلومات من مصدر خارجي وتوليد النص بناءً عليها.  |
| Workflow                           | تسلسل منظم من الخطوات لإنجاز مهمة معينة.                               |
| Automation                         | أتمتة المهام المتكررة لتقليل التدخل اليدوي.                            |

| المصطلح                        | الشرح                                                                                   |
|--------------------------------|-----------------------------------------------------------------------------------------|
| <b>Software Architecture</b>   | البنية العامة لنظام برمجي وعلاقات مكوناته.                                              |
| <b>AI-Assisted Development</b> | تطوير البرمجيات بالاستعانة بأدوات الذكاء الاصطناعي كمساعد، دون تسليم القرار الكامل لها. |



## المراجع الرسمية (Official References)

للاطلاع على أحدث المعلومات الدقيقة حول Claude و Claude Code وواجهة برمجة التطبيقات (API)، يُرجى دائماً الرجوع إلى المصادر الرسمية التالية بدلاً من الاعتماد فقط على محتوى هذا الكتاب:

[التوثيق الرسمي لـ Claude و API — https://docs.claude.com](https://docs.claude.com)

[مركز الدعم الرسمي لـ Claude.ai — https://support.claude.com](https://support.claude.com)

[الموقع الرسمي لشركة Anthropic — https://www.anthropic.com](https://www.anthropic.com)

[أخبار وتحديثات المنتجات الرسمية — https://www.anthropic.com/news](https://www.anthropic.com/news)

[حزمة Claude Code الرسمية على npm — https://www.npmjs.com/package/@anthropic-ai/claude-code](https://www.npmjs.com/package/@anthropic-ai/claude-code)

عن المؤلفة

تسنيم نمر نصر

***Tasneem Nimr Nasr — Software Engineering  
Student***

تسنيم نمر نصر طالبة هندسة برمجيات مهتمة بتطوير البرمجيات، وأدوات الذكاء

الاصطناعي، وطرق دمج AI مع عملية Software Development.

هذا الكتاب هو مشروع شخصي وتقني من إعدادها، تشارك فيه ما تعلمته حول استخدام

Claude و Claude Code بطريقة احترافية ومسؤولة.